

**Build Restful API to execute CRUD Operations case study: Suggested Case Study: University Library RESTful API for Book Management** A university library intends to develop a RESTful API to efficiently manage its book inventory. The API will enable users, including students and librarians, to add, retrieve, update, and delete books from the system. Each book entry will contain essential details such as title, author, ISBN, publication year, and availability status. **Functional Requirements:** The API should be implemented using Spring Boot and must support the following CRUD operations: 1. GET /books – Retrieve a list of all books in the library. 2. GET /books/{id} – Fetch details of a specific book based on its ID. 3. POST /books – Add a new book to the library, with the request body containing book details. 4. PUT /books/{id} – Update an existing book's details, such as availability status. 5. DELETE /books/{id} – Remove a book from the library system.

```
package com.example.Book.Management.service;

import com.example.Book.Management.document.Book;
import com.example.Book.Management.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

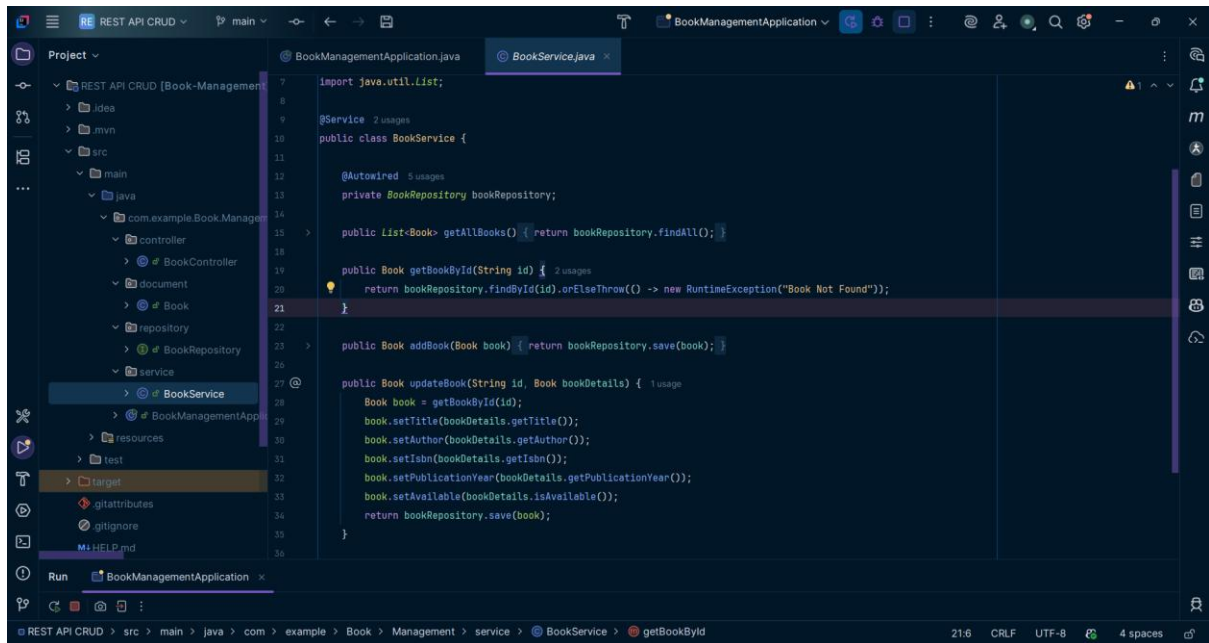
    public Book getBookById(String id) {
        return bookRepository.findById(id).orElseThrow(() -> new RuntimeException("Book Not Found"));
    }

    public Book addBook(Book book) {
        return bookRepository.save(book);
    }

    public Book updateBook(String id, Book bookDetails) {
        Book book = getBookById(id);
        book.setTitle(bookDetails.getTitle());
        book.setAuthor(bookDetails.getAuthor());
        book.setIsbn(bookDetails.getIsbn());
        book.setPublicationYear(bookDetails.getPublicationYear());
        book.setAvailable(bookDetails.isAvailable());
        return bookRepository.save(book);
    }

    public void deleteBook(String id) {
        bookRepository.deleteById(id);
    }
}
```

```
}  
}  
}
```



```
import java.util.List;  
  
@Service 2 usages  
public class BookService {  
  
    @Autowired 5 usages  
    private BookRepository bookRepository;  
  
    public List<Book> getAllBooks() { return bookRepository.findAll(); }  
  
    public Book getBookById(String id) { 2 usages  
        return bookRepository.findById(id).orElseThrow(() -> new RuntimeException("Book Not Found"));  
    }  
  
    public Book addBook(Book book) { return bookRepository.save(book); }  
  
    public Book updateBook(String id, Book bookDetails) { 1 usage  
        Book book = getBookById(id);  
        book.setTitle(bookDetails.getTitle());  
        book.setAuthor(bookDetails.getAuthor());  
        book.setIsbn(bookDetails.getIsbn());  
        book.setPublicationYear(bookDetails.getPublicationYear());  
        book.setAvailable(bookDetails.isAvailable());  
        return bookRepository.save(book);  
    }  
}
```